problem1 - SCC and Reversal

1. Algorithm to compute rev(G)

We create a new graph rev(G):

For each vertex, keep it the same

for each $(u \to v)$, reverse it to $(v \to u)$

Since we process every vertex and edge once, the time complexity is: $O(V+E)$

2. proves SCC(G) is acyclic

Assume SCC(G) has acycles

$C_1 \to C_2 \to \dots \to C_k \to C_1$

Then all these components can reach each other so they should actually be one single strongly connected component.

This is a contradiction

Therefore, SCC(G) is acyclic (a DAG)

3 prove $scc(rev(G)) = rev(scc(G))$

Reversing all edges does not change strongly connected components

However, edges between components are reversed.

therefore; $SCC(rev(G)) = rev(SCC(G))$

4. Reachability relation

we prove:

u can reach v $\Leftrightarrow$ $S(u)$ can reach $S(v)$

($\Rightarrow$)

if $u \to v$, then their components also have a path:

$S(u) \to S(v)$

($\Leftarrow$)

if $S(u) \to S(v)$, then there is a path between components

so $u \to v$

Hence, the statement is true.

Problem 2 → Euler Tour

condition for existence

A directed graph has an Euler tour iff:

for every vertex v:

in - degree(v) = out - degree(v)

Reason:

Every time we enter a vertex, we must leave it.

2 O(E) algorithm

simple idea

1. Start from any vertx
2. Follow unused edges to form a cycle
3. If ~~unu~~ unused edges remain:

   (1) start from a vertex in the cycle

   (2) Build another cycle.
4. merge all cycles.

Each edge is used once → O(E)

problem 3 - Topological sort

Graph edges:

A → B, A → C, B → C, B → D, C → E, D → E, D → F

G → F, G → E.

1. starting from A

one valid order:

A → B → C → D → E → F → G

Another:

A → B → D → C → E → F → G

2. starting from another node (e.g. G)

G → A → B → D → F → C → E or

G → A → B → C → D → F → E

Note

Topological order is not unique Any order that respects edge directions is correct.